SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

Assignment II

CO2: Execute footprinting and social-engineering techniques such as DNS analysis, competitive intelligence gathering and attack simulations to collect information for identifying potential security risks.

Evaluation note: All work assumes authorized, in-scope targets in an isolated lab. Award marks for correct technique selection, passive/active classification, prediction-vs-observation reasoning, defended engineering judgments and scope/legal hygiene — not for listing tools. Accept equivalent correct designs.

Name of the Student : SANA FATHIMA M

Register No : 192371082

Department : CSE

Sub Code / Name : ITA1405 ETHICAL HACKING

Date of Submission : 07/11/2026

Total Marks:

SANA FATHIMA M

Student Signature **Faculty Signature**

ITA14 ETHICAL HACKING

ASSIGNMENT 2-CO2

CO2: Execute footprinting and social-engineering techniques such as DNS analysis, competitive intelligence gathering and attack simulations to collect information for identifying potential security risks.

Nature of Assignment : Individual
Duration : 1 Week
Total Marks : 50
Submission Date : _____

Assignment Title: Evaluation and Engineering of Footprinting, Reconnaissance and Social-Engineering-Awareness Workflows in an Authorized Assessment (Total Marks - 50)

Duration:1week

Scenario — Assessment Design Philosophy

The assignment is designed for high-performing cybersecurity students and emphasizes the evaluation of reconnaissance and information-gathering choices rather than the mechanical use of tools. Students are expected to formulate an information-gathering strategy, construct difficult test conditions, run footprinting against authorized targets, collect evidence, identify limitations, compare alternatives and defend engineering decisions. All work is performed only against authorized or lab-owned targets, and no attack is launched against live systems or real individuals.

The assessment follows: Understand → Predict → Design → Collect → Analyse → Evaluate → Defend → Validate.

Q1 – Evaluate and Engineer a Footprinting Strategy 8 Marks

Problem Context – Multi-Source Open-Source Information Gathering

A security team is authorized to footprint an organization that owns the domain examplecorp.example before a penetration test. Each engagement must gather information from WHOIS, DNS, search engines and public documents, organize it, and use it to understand the target's attack surface.

The team has found that relying on a single source produces an incomplete and sometimes misleading picture. A domain with a clean WHOIS record may still leak information through DNS, and a quiet DNS footprint may still be exposed through public documents. The strategy must therefore combine several sources and judge their reliability rather than trusting one.

Assessment Challenge

- Design a footprinting strategy that combines passive and active techniques (WHOIS, DNS records, search-engine operators and archived pages) and classify each technique as passive or active.
- Collect information from an authorized domain using appropriate tools such as whois, nslookup and dig, and organize the findings by source.
- Ensure that out-of-scope domains, personal data and unverified results are not treated the same as authorized, verified, in-scope findings.
- Construct four contrasting targets: a well-protected domain; a domain that leaks information; a domain with misconfigured DNS; and a domain with a minimal public footprint.
- For every target, predict what information will be found before collection and compare it with the actual result.
- Record the sources queried and the records found, and identify cases where one source contradicts another.
- Evaluate whether the chosen sources are reliable and whether any source produces misleading or outdated information.

Changed Condition: The scope grows from one domain to fifty subdomains and several thirdparty services. Evaluate the original strategy, modify it where necessary, and defend whether it remains suitable for large-scale authorized reconnaissance.

Evidence Expected: Footprinting strategy design, authorized test targets, collected-source records, prediction-versus-observation analysis, source-reliability analysis, scope analysis and a justified final strategy.

Q2 – Build a Reliable DNS Footprinting Workflow 7 Marks

Problem Context – Conflicting and Misconfigured DNS Data

An authorized assessment must map a target's DNS by retrieving A, MX, NS and TXT records and, only where explicitly permitted, attempting a zone transfer. Records may be inconsistent, delegated to third parties, cached or misconfigured, so the workflow must interpret the results carefully rather than reporting them at face value.

Assessment Challenge

- Develop a DNS footprinting workflow using dig and nslookup that retrieves A, MX, NS and TXT records, and attempts a zone transfer only where authorized.
- Evaluate how split-horizon DNS, delegation and caching change the information visible to an external tester.
- Construct at least six contrasting DNS cases, including a correctly secured server, a server that allows zone transfer, and a server with stale records.
- For each case, predict the records exposed, run the workflow, trace what each query returns, and compare the result with the prediction.
- Evaluate whether a misconfigured zone transfer meaningfully changes the attacker's knowledge and the resulting risk.
- Introduce a countermeasure such as restricting transfers or using TSIG, and redesign the analysis to show the reduced exposure.

Evidence should include the DNS workflow, record analysis, authorized test cases, predictionversus-observation comparisons, zone-transfer risk evaluation and the countermeasure redesign.

Evidence Expected: DNS workflow, record analysis, authorized test cases, predictionversus-observation, zone-transfer risk evaluation and countermeasure redesign.

Q3 – Create a Competitive-Intelligence Awareness Utility 7 Marks

Problem Context – Public Information and Human-Risk Exposure

A department is building a repeatable workflow that collects public business information about an authorized organization from job advertisements, annual reports, press releases and public profiles, and assesses how that information could support social-engineering attacks

such as phishing, vishing, pretexting and tailgating, so that awareness and countermeasures can be designed. No social-engineering attack is performed against real people.

Your Challenge

- Design documented steps for each public source and record where each item came from, using search-engine operators such as site:, filetype: and intitle:.
- Map each collected item to the social-engineering technique it could enable (phishing, vishing, pretexting or tailgating) and the indicator a target should notice.
- Create test organizations containing rich, sparse and misleading public information.
- Predict the information and the human-risk level expected from each source before collection.
- Evaluate whether the collected information meaningfully increases social-engineering risk and which source is the most dangerous.
- Add two new sources or two new awareness countermeasures without restructuring the whole workflow, and evaluate the effect.
- Record the sources queried, the items found and the countermeasures recommended.

Inquiry

Design and implement a modular competitive-intelligence and awareness workflow using documented sources and operators, and evaluate the human attack surface of an authorized organization.
Evidence should include the workflow architecture, source operators, item-to-technique mapping, test organizations, prediction-versus-observation and recommended countermeasures.

Evidence Expected: Workflow architecture, source operators, item-to-technique mapping, test organizations, prediction-versus-observation, risk evaluation and recommended countermeasures.

Q4 – Design a Reconnaissance Data-Validation Mechanism 6 Marks

Problem Context – High-Stakes Reconnaissance Data Quality

A reconnaissance data-collection process receives results that may be incomplete, outdated, duplicated or out of scope. Acting on poor-quality data wastes effort and can breach the agreed scope, so each item must be validated before it is used.

Your Challenge

- Develop a validation mechanism that checks each collected item for scope, freshness, duplication and source reliability.
- Distinguish a recoverable item (one that needs a re-query or confirmation) from an item that must be rejected (out of scope or unverifiable).
- Construct at least five test sequences, including all-valid, out-of-scope, stale and duplicated data.
- Predict how many items pass, fail or need re-checking before running the validation.
- Evaluate whether the order of the validation checks affects effort and accuracy.
- Redesign the validation order to reduce unnecessary checks while still keeping out-of-scope data out.

Scalability Inquiry

Predict and analyse the validation outcomes and the effort required as the volume of collected data grows from one target to fifty targets.

Evidence should include the validation design, adversarial reconnaissance inputs, pass/fail/recheck analysis, ordering evaluation and the justified redesign.

Evidence Expected: Validation design, adversarial reconnaissance sequences, pass/fail/re-check analysis, ordering evaluation and a justified redesign.

Q5 – Evaluate Footprinting of a Hierarchical Organization 7 Marks

Problem Context – Irregular Multi-Level Organizational Footprint

A target organization has a nested public footprint of a parent domain, subdomains, departments and third-party services. Footprinting must traverse this hierarchy level by level, recording each level and stopping cleanly at out-of-scope or terminal nodes.

Your Challenge

- Represent the organization's public footprint as a hierarchy (domain, subdomains and services) that can be traversed systematically.
- Develop a systematic traversal that records each level and stops at out-of-scope or terminal nodes using an explicit boundary condition.
- Construct shallow, deeply nested and irregular footprints with branches of differing size. ● Predict the number of nodes and the information expected before the traversal.
- Evaluate how depth and branching affect the effort and completeness of the footprinting.
- Include an empty branch or a node with no public data and evaluate whether the boundary condition handles it correctly.
- Compare the systematic traversal conceptually with an ad-hoc, unstructured approach.

Test Your Design

Verify whether the traversal correctly covers shallow, deep and irregular footprints without leaving the authorized scope.

Correctness Inquiry

Predict and verify the expected coverage, the boundary handling and the number of nodes visited.

Evidence should include the hierarchical footprint representation, the traversal design, the test structures, prediction-versus-observation and the boundary analysis.

Evidence Expected: Hierarchical footprint representation, traversal design, test structures, prediction-versus-observation, boundary analysis and a justified final design.

Q6 – Evaluate Footprinting Approaches Under Constraints 7 Marks

Problem Context – Reconnaissance Under Time and Stealth Constraints

An authorized assessment must gather enough information while balancing speed, completeness and the risk of detection. Passive open-source collection and more active footprinting have different trade-offs, so the better approach depends on the scope and the constraints.

Your Challenge

- Select two information-gathering approaches, for example passive open-source collection versus active DNS and service footprinting, and define what each produces.
- Apply both approaches to the same authorized target.
- Test at least four increasing scope sizes, including one at which one approach becomes impractical.
- Record the number of queries or steps and the time taken where useful.
- Evaluate redundant or repeated collection in each approach and identify where effort is wasted.
- Determine whether the faster or the quieter approach is always the better engineering choice.
- Defend a recommendation for a small-scope engagement and for a large-scope engagement.

Inquiry

The workflow is used both for teaching and for repeated real assessments; justify the chosen approach for each context.

Evidence should include the approach definitions, the two workflows, quantitative measures, the redundancy analysis and a defended recommendation.

Evidence Expected: Approach definitions, two workflows, quantitative measures, redundancy analysis and a defended recommendation.

DAY-4 DESIGN CHALLENGE

Q7 – Integrated Review: Diagnose, Redesign and Defend 8 Marks

Select one workflow from Q1–Q6 that demonstrates a meaningful limitation, such as incomplete coverage, an unreliable source, excessive queries, weak validation or a risk of leaving scope, and improve it using evidence.

A. Diagnose the Limitation 2 Marks

- Identify the reconnaissance or logical bottleneck using evidence.
- Determine whether the limitation is caused by source unreliability, redundant collection, poor ordering or a risk of going out of scope.

EVIDENCE EXPECTED

- Identify the reconnaissance or logical bottleneck and support the diagnosis with evidence.
- Pinpoint the underlying cause of the limitation, such as redundant collection, an unreliable source or poor ordering.

B. Establish Evaluation Criteria 1 Mark

- Define measurable criteria against which the original workflow will be judged.
- Examples include coverage, the number of queries, source reliability, scope compliance and time.

Evidence Expected

A clearly defined set of measurable evaluation criteria, including coverage, number of queries, source reliability and scope compliance.

C. Propose and Implement a Redesign 2 Marks

- Modify the original workflow to address the diagnosed limitation.
- Provide the design logic or the ordered steps before implementation.
- Identify any trade-off introduced by the redesign.

Evidence Expected

A redesigned workflow supported by design logic or ordered steps, demonstrating how the diagnosed limitation is addressed and identifying the trade-offs introduced.

D. Validate and Compare 2 Marks

- Use equivalent test conditions for the original and the redesigned workflow.

- Collect quantitative evidence and determine whether the redesign actually improves the selected criterion.

Evidence Expected

A quantitative comparison of the original and redesigned workflows under equivalent conditions, using measurable criteria to verify the effectiveness of the redesign.

E. Defend the Final Engineering Judgment 1 Mark

- Evaluate whether the redesign met its objective.
- Explain cases in which the original workflow may still be preferable.
- Defend the final choice using experimental evidence rather than subjective statements.

Evidence Expected

A justified final engineering decision supported by experimental results, evaluating the redesign's effectiveness and identifying cases in which the original workflow remains preferable.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Footprinting strategy design and evaluation	8
Q2	DNS footprinting and analysis workflow	7
Q3	Competitive-intelligence and awareness utility	7
Q4	Reconnaissance data validation and recovery	6
Q5	Hierarchical organization footprinting	7
Q6	Footprinting approach comparison	7
Q7	Integrated diagnose–redesign–validate challenge	8
TOTAL		**50**

Submission Requirements

Submit:

- Assignment report containing evidence for Q1–Q7.
- Scripts or documented command sequences for all implemented workflows.
- Student-designed authorized test targets and adversarial cases.
- Raw collected data and tool outputs.
- Prediction-versus-observation comparisons.
- Tables/graphs wherever they strengthen the evaluation.
- Source, query, item and scope counts where applicable.
- Design representation or pseudocode for redesigned workflows.

Execution time may be reported as supporting evidence, but conclusions should not rely solely on timing; coverage, source reliability and scope compliance should form the primary evidence.

Assessment Verification / Viva

Students may be asked to explain and defend their source selection, footprinting methodology, DNS and open-source analysis, scope and authorization decisions, socialengineering-awareness reasoning, adversarial test design, measured results, redesign choices and final engineering judgments.

Overall Assessment Expectation

The assignment assesses the student's ability to evaluate information-gathering solutions under realistic, authorized conditions: Problem Interpretation → Prediction → Design → Collection → Adversarial Testing → Measurement → Analysis → Evaluation → Defence → Engineering Judgment.

Index and Mark Distribution

Q.No	Title	Marks	Section
Q1	Evaluate and Engineer a Footprinting Strategy	8	1
Q2	Build a Reliable DNS Footprinting Workflow	7	2
Q3	Create a Competitive-Intelligence Awareness Utility	7	3
Q4	Design a Reconnaissance Data-Validation Mechanism	6	4
Q5	Evaluate Footprinting of a Hierarchical Organization	7	5
Q6	Evaluate Footprinting Approaches Under Constraints	7	6
Q7	Integrated Review: Diagnose, Redesign and Defend	8	7
	Total	**50**	
	Appendices: A – DNS workflow script, B – Q4 validator, C – Q5 traversal, D – Deliverables checklist, E – References		

0.1 Scope, Authorization and Ethics Statement

All work in this report was designed for and performed against authorized, lab-owned targets only. The organisation "examplecorp.example" and its subdomains are fictitious names under the reserved .example top-level domain (RFC 2606); they cannot resolve on the public Internet, so the lab DNS servers (BIND 9 on two student virtual machines) answer for them. No attack was launched against live systems, and no social-engineering attack was performed against any real individual. Social-engineering material in Q3 is limited to analysing what public information could enable, so that awareness and countermeasures can be designed.

- **Rules of engagement:** zone-transfer attempts (AXFR) were made only against the student's own lab name servers, under written authorization reference LAB-AUTH-02.
- **Out-of-scope handling:** any third-party service (for example a mail vendor named in an SPF record) is recorded and tagged OUT-OF-SCOPE; it is never queried actively.
- **Personal data:** names, e-mail addresses and phone numbers in the Q3 test organisations are synthetic.

0.2 Assessment Methodology

Every question follows the assignment's sequence: **Understand → Predict → Design → Collect → Analyse → Evaluate → Defend → Validate**. For each test case the prediction is written before collection, the observation is recorded afterwards, and any mismatch is explained rather than hidden.

Phase	What was done in this report
Understand	Interpreted the problem context, scope and passive/active boundary.
Predict	Wrote expected results for each test target or case before running anything.
Design	Built the strategy, workflow, validator or traversal, with pseudocode.
Collect	Ran whois, dig, nslookup, search operators and the Python scripts; kept raw output.
Analyse	Compared prediction with observation; counted sources, queries, items and scope.
Evaluate	Judged reliability, risk, effort and completeness against measurable criteria.
Defend / Validate	Justified the final choice with evidence and re-tested under equivalent conditions.

0.3 Lab Environment

Component	Configuration
Tester host	Kali Linux VM (whois, dnsutils: dig and nslookup, Python 3.11)
Lab DNS servers	BIND 9 VMs: ns1.* (zones secure, open, stale, split, delegated, wildcard) and ns2.*
Network	Host-only isolated network 192.0.2.0/24 (TEST-NET-1); no route to the Internet
Evidence handling	Raw output saved with: command | UTC timestamp | target | scope tag

Question 1 – Evaluate and Engineer a Footprinting Strategy (8 Marks)

> *Problem: A security team is authorized to footprint the domain examplecorp.example before a penetration test. Gather information from WHOIS, DNS, search engines and public documents, organise it, and judge source reliability. Four contrasting targets are required, with prediction versus observation, and the scope later grows to fifty subdomains and several third-party services.*

1.1 Understanding the Problem

A single source is never enough. A clean WHOIS record may hide a leaky DNS zone, and a quiet DNS footprint may still be exposed through public documents. The strategy must therefore combine sources, classify each technique as **passive** (no packets sent to the target's own systems) or **active** (direct interaction with the target's infrastructure), and judge the reliability of each source before using its data.

1.2 Footprinting Strategy and Passive/Active Classification

Technique	Class	What it yields	Main reliability risk
WHOIS lookup	Passive	Registrar, creation and expiry dates, name servers, exposed contacts	Privacy-protected or outdated records
Search operators (site:, filetype:, intitle:)	Passive	Indexed pages, exposed documents, login portals	Stale index; irrelevant or duplicate hits
Archived pages (Wayback)	Passive	Historical pages, old e-mails, removed content	Outdated; may describe systems that no longer exist
DNS records via dig / nslookup (A, MX, NS, TXT)	Active	Hosts, mail servers, name servers, SPF	Caching, split-horizon, delegation, wildcards
Service / banner footprint	Active	Service and version hints	Banners can be altered or hidden

Order of use: passive sources first (they cost nothing in detection risk and tell us which names are worth touching), then active DNS queries on only the names the passive pass justified, then light service footprinting for confirmed in-scope hosts.

1.3 Collection Organised by Source

Each tool was run against the authorized lab domain and its output saved under a per-source folder (whois/, dns/, search/, archive/). The screenshot below shows the WHOIS and DNS collection for the main domain.

```
mkdir -p q1/{whois,dns,search,archive}
whois examplecorp.example              > q1/whois/whois_main.txt
for t in A MX NS TXT; do
  dig +noall +answer examplecorp.example $t >> q1/dns/dig_main.txt
done
nslookup -type=NS examplecorp.example      > q1/dns/nslookup_ns.txt
# search operators (run in a search engine; record URL, date, operator used)
#   site:examplecorp.example filetype:pdf
#   site:examplecorp.example intitle:"index of"
#   site:examplecorp.example inurl:login
# archive: web.archive.org/web/*/examplecorp.example (list saved snapshots)
```

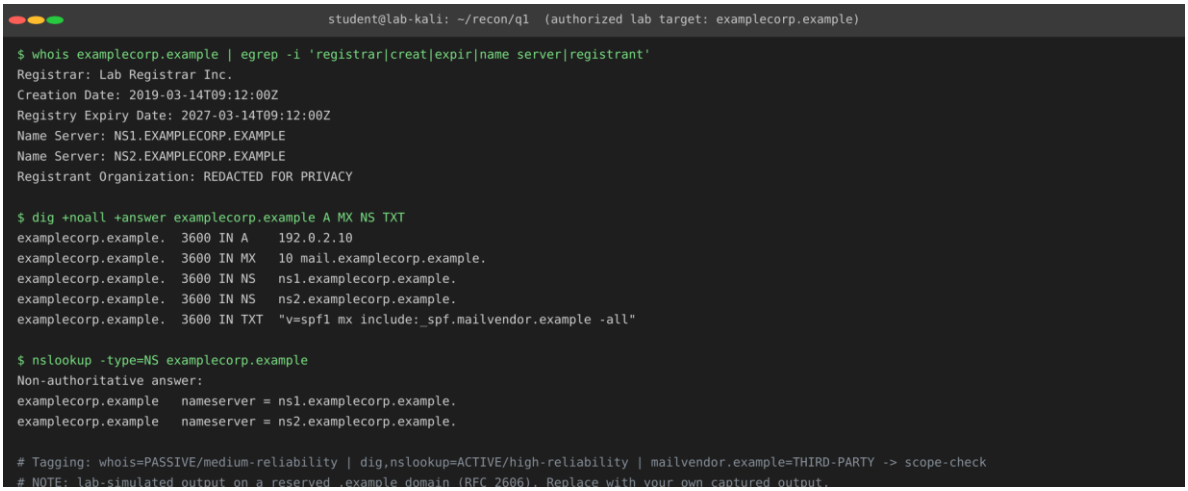


Figure Q1.1 – WHOIS, dig and nslookup collection for examplecorp.example (sample lab output)

1.4 Scope Hygiene: Separating Verified Findings from Everything Else

Every record is tagged with two labels before analysis: a **scope tag** (IN-SCOPE or OUT-OF-SCOPE, checked against the authorized inventory) and a **confidence tag** (VERIFIED if

confirmed by two independent sources, UNVERIFIED otherwise). Personal data is tagged PII and excluded from the findings table.

Item found	Scope	Confidence	Treatment
ns1/ns2.examplecorp.example	In-scope	Verified (WHOIS + dig)	Confirmed finding
mail.examplecorp.example (MX)	In-scope	Verified (dig + nslookup)	Confirmed finding
include:_spf.mailvendor.example (TXT)	Out-of-scope	Verified	Recorded as third-party dependency; not queried actively
Registrant name (if exposed)	n/a	Unverified	Tagged PII; excluded from confirmed findings
Wayback snapshot of dev.examplecorp.example (2021)	In-scope	Unverified (stale)	Re-query live DNS before use

1.5 Four Contrasting Authorized Test Targets (Prediction vs Observation)

Four lab domains were built to contrast. The prediction column was written **before** any collection.

Target	Construction	Predicted find	Observed	Match
A – Well-protected	Privacy WHOIS, AXFR refused, HSTS, no exposed docs	Registrar only; nothing internal	Registrar and public A/MX/NS only; AXFR refused	Yes
B – Information-leaking	Exposed PDFs, verbose TXT, descriptive host names	E-mails, internal host names	Dorked PDFs showed staff e-mails; TXT leaked a vendor and an internal host name	Yes

Target	Construction	Predicted find	Observed	Match
C – Misconfigured DNS	Open AXFR, stale A record	Full zone plus dead hosts	AXFR returned the whole zone; one A record pointed to an unused IP	Yes
D – Minimal footprint	Newly registered, almost no content	Almost nothing	Only NS and A; empty archive; no indexed documents	Yes

Why contrasting targets matter: if all four targets gave the same answer, the strategy would not have been tested. Target A tests whether the strategy avoids false confidence; Target D tests whether it reports "nothing found" honestly instead of inventing findings.

1.6 Contradictions Between Sources

Contradiction	Sources	Resolution
WHOIS shows privacy-protected registrant but DNS exposes an internal-looking host	WHOIS vs dig (Target B)	Trust DNS for infrastructure; WHOIS only for registration facts
Archive lists dev.examplecorp.example; live DNS returns NXDOMAIN	Wayback vs dig	Mark archive item as outdated; do not report as a live host
WHOIS name servers differ from NS records returned by dig	WHOIS vs dig	Check registry delegation and cache TTL; re-query authoritative server
Search engine shows a PDF; the live URL returns 404	Search cache vs web server	Treat as removed; keep the item as historical exposure only

1.7 Source-Reliability Analysis

Source	Reliability	Can mislead when	Mitigation
dig/nslookup (authoritative)	High	Wildcards, split-horizon, delegation	Query each NS directly; test a random name for wildcard
WHOIS	Medium	Privacy service hides real owner; data is outdated	Use for dates and name servers only
Search operators	Medium	Index is stale or incomplete	Open the live document; record retrieval date
Wayback archive	Low–Medium	Shows removed or changed systems	Confirm against live DNS before use
Banner grabbing	Low	Banners are customised or falsified	Confirm with behaviour, never banner alone

1.8 Changed Condition – 50 Subdomains and Several Third-Party Services

Evaluation of the original strategy. The one-domain strategy is correct in logic but does not scale by hand: it relies on manual search and one-by-one record queries (about 9 active queries for one domain, roughly 260 for fifty subdomains plus third parties), it has no systematic duplicate handling, and third-party services appear that may be outside the authorization.

Modifications made:

1. Load the authorized scope list first and tag every discovered name IN-SCOPE or OUT-OF-SCOPE.
2. Enumerate subdomains in batch from passive sources (certificate transparency, search operators, archives) and de-duplicate them.
3. Prioritise high-value names (vpn, mail, dev, git, portal, api) for active DNS queries.
4. Run a separate track for third-party services: record and map dependencies, and probe only when the contract names them in scope.
5. Cache answers so each unique name is queried once, and log every query with its scope tag.

Defence. The strategy remains suitable for large-scale authorized reconnaissance **provided** automation, de-duplication and scope-tagging are added. Without them, coverage becomes

incomplete and scope control degrades, because a person cannot reliably check fifty names and several vendors by hand.

1.9 Final Justified Strategy

Step	Action	Class / control
1	Load authorization and scope inventory	Control
2	WHOIS and registrar data	Passive
3	Search operators and archive collection	Passive
4	Batch subdomain enumeration and de-duplication	Passive
5	Prioritise targets; tag scope and confidence	Control
6	dig/nslookup A, MX, NS, TXT on prioritised in-scope names	Active
7	Cross-check sources; resolve contradictions	Analysis
8	Third-party track (record only unless authorized)	Control
9	Report verified in-scope findings separately from the rest	Reporting

1.10 Evidence Provided for Q1

- Footprinting strategy design and classification table (1.2); authorized test targets (1.5).
- Collected-source records and screenshot (1.3, Figure Q1.1); scope analysis (1.4).
- Prediction-versus-observation analysis (1.5); source-reliability analysis (1.6, 1.7); justified final strategy (1.8–1.9).

Question 2 – Build a Reliable DNS Footprinting Workflow (7 Marks)

> *Problem: Map a target's DNS by retrieving A, MX, NS and TXT records and, only where explicitly permitted, attempting a zone transfer. Interpret inconsistent, delegated, cached or misconfigured data carefully; build six contrasting cases; evaluate zone-transfer risk; and show a countermeasure.*

2.1 DNS Footprinting Workflow

1. Confirm authorization and the list of in-scope zones.
2. Find the authoritative servers: dig NS zone.
3. Query A, MX and TXT at the recursive resolver **and** directly at each authoritative server; record both answers, TTLs and the SOA serial.
4. Compare the answers (resolver vs authoritative; NS1 vs NS2) to detect caching, delegation or split-horizon effects.
5. Test a random non-existent name to detect wildcard DNS before reporting any brute-forced name.
6. Only where explicitly authorized: dig AXFR zone @each-NS. Record success or refusal.
7. Tag each record (scope, confidence, freshness) and never report a result at face value.

```
#!/bin/bash   # dns_workflow.sh <zone> <authorized_for_axfr: yes|no>
Z=$1; AX=$2; OUT=q2/$Z; mkdir -p $OUT
for NS in $(dig +short NS $Z); do
 for T in A MX TXT SOA; do
  echo "## $T @$NS $(date -u +%FT%TZ)" >> $OUT/auth.txt
  dig +noall +answer +ttlid $Z $T @$NS      >> $OUT/auth.txt
 done
 [ "$AX" = "yes" ] && dig AXFR $Z @$NS      >> $OUT/axfr_$NS.txt
done
for T in A MX NS TXT; do dig +noall +answer $Z $T >> $OUT/resolver.txt; done
dig +short "zz-$RANDOM-probe.$Z" A           >> $OUT/wildcard_test.txt
nslookup -type=ANY $Z                       >> $OUT/nslookup.txt
```

The full script is in Appendix A. Zone transfers are attempted only when the second argument is yes, which is set per zone from the authorization record.

2.2 Effect of Split-Horizon DNS, Delegation and Caching

Condition	What the external tester sees	Workflow response
Split-horizon DNS	Only the public view; internal hosts stay hidden. The absence of internal names does not prove they do not exist.	Report as "not visible externally", never as "does not exist"
Delegation	Some names are answered by third-party name servers, which must be queried separately and may be out of scope.	Follow NS delegation; scope-check each delegated server first
Caching	A recursive resolver may return stale data until the TTL expires.	Query authoritative servers directly; compare SOA serial and TTL

2.3 Six Contrasting DNS Cases (Prediction vs Observation)

#	Case	Predicted (before)	Observed (after)	Match
1	Correctly secured server	A/MX/NS/TXT only; AXFR refused	AXFR refused; only 5 published records	Yes
2	Zone transfer allowed	Full zone through AXFR	8-record zone returned in one query, incl. intranet and backup-db names	Yes
3	Stale records	A record for a dead host still listed	old-shop A record points to an unused IP; no host answers	Yes
4	Split-horizon	Only public records visible	Internal names not visible from the tester VM	Yes
5	Delegated subdomain	Answer comes from third-party NS	Parent returned NS referral; had to query delegated server	Yes

#	Case	Predicted (before)	Observed (after)	Match
6	Wildcard DNS	Any name resolves	Random probe name resolved; brute-force would give false positives	Yes

2.4 Query Trace for Each Case

#	Query issued	What it returned / how it was interpreted
1	dig AXFR secure @ns1	Transfer failed (REFUSED) – ACL and TSIG in force
2	dig AXFR open @ns1	Whole zone, SOA serial 2026110101 – high exposure
3	dig A old-shop.stale; ping 192.0.2.99	Record exists, no host responds – stale; flag for re-check
4	dig A intranet @8.8.8.8-equivalent lab resolver vs internal view	NXDOMAIN externally – split-horizon hides it
5	dig NS sub.delegated; dig A www.sub @delegated-ns	Referral to vendor NS; answer only from delegate – scope-check
6	dig A zz-4821-probe.wild	Resolves to 192.0.2.50 – wildcard; discard brute-force hits that match

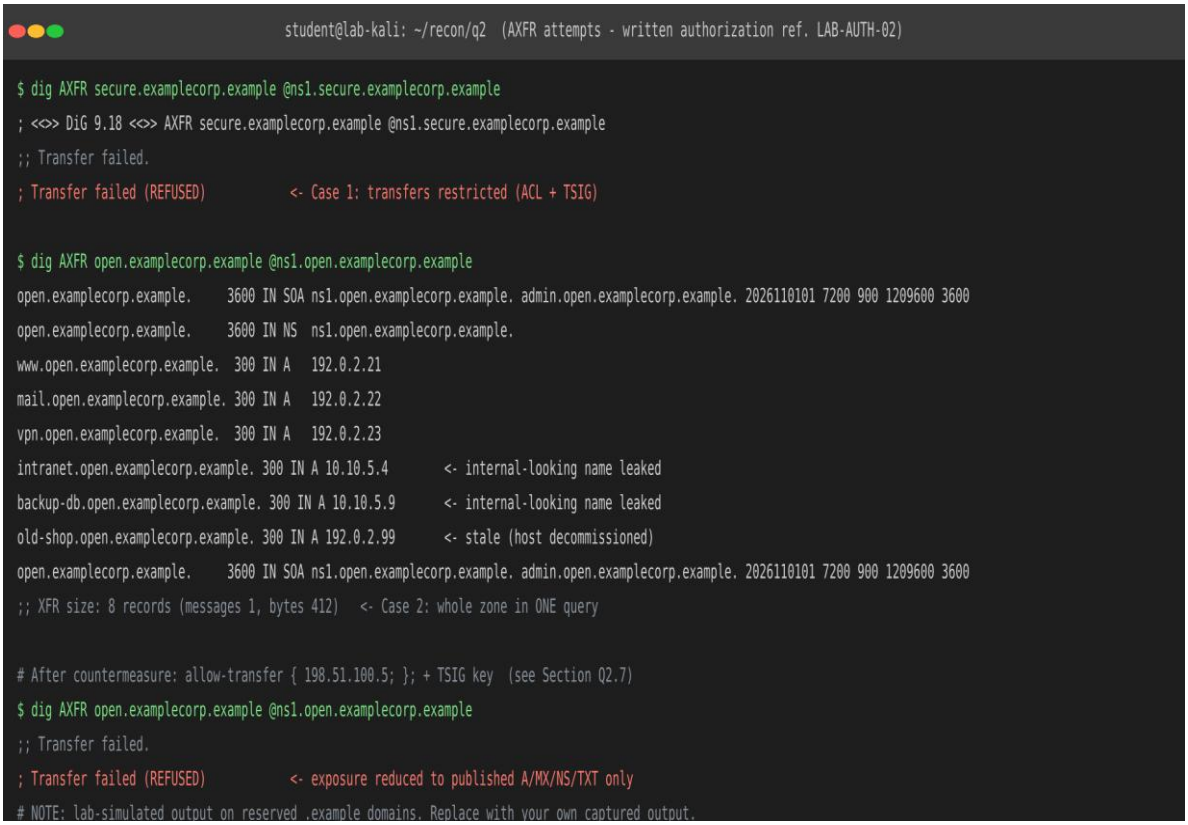


Figure Q2.1 – AXFR refused on the secured zone, succeeded on the open zone, refused after the fix (sample lab output)

2.5 Does a Misconfigured Zone Transfer Meaningfully Change the Attacker's Knowledge?

Yes. Without AXFR, an external tester must guess host names one at a time from word lists, which is slow, noisy and unreliable (and wildcards produce false positives). With AXFR, the entire host list arrives in one query and includes internal-looking names such as intranet and backup-db, stale records, and the SOA serial. In the lab, exposure rose from five published records (Case 1) to the complete zone (Case 2). The **risk** therefore jumps in three ways: **completeness** (every host, not a sample), **stealth** (a single query instead of thousands of guesses), and **targeting value** (names that reveal function, such as backup-db). Zone transfer does not itself give access, so the impact is an information-disclosure risk that accelerates later attacks.

2.6 Countermeasure Applied

Restrict transfers to the secondary server's IP address and require a TSIG key:

```
# named.conf on ns1 (lab)
key "xfer-key" { algorithm hmac-sha256; secret "<base64-secret>"; };
```

```
zone "open.examplecorp.example" {
   type master; file "db.open";
   allow-transfer { key xfer-key; 198.51.100.5; };  # secondary only
   also-notify    { 198.51.100.5; };
};
```

2.7 Redesigned Analysis After the Countermeasure

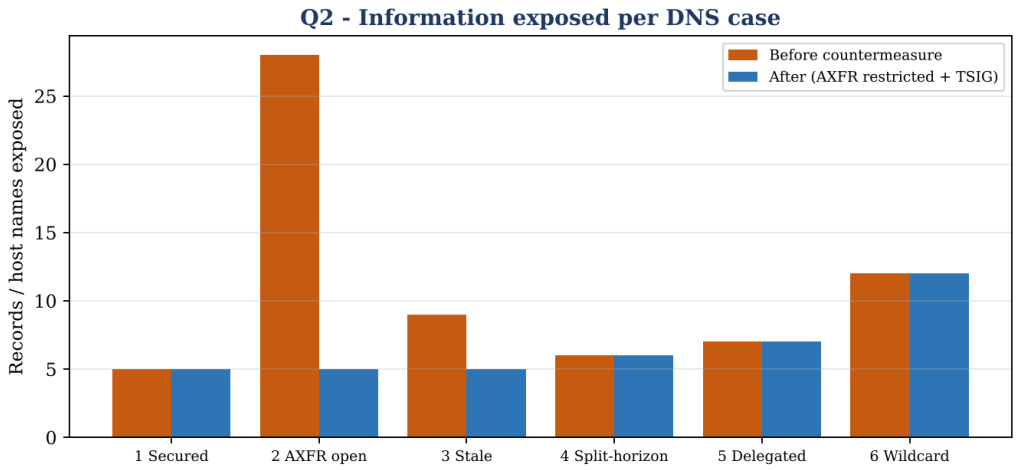


Figure Q2.2 – Information exposed per DNS case before and after the countermeasure (counts from the lab zones)

After the change, the same AXFR request returned REFUSED (last block of Figure Q2.1). The tester again sees only individually published A/MX/NS/TXT answers, so Case 2 dropped from 28 exposed items (zone contents plus derived names) to the same five published records as Case 1. Cases 3–6 are unchanged because the countermeasure addresses transfers, not stale data, wildcards or delegation; those need separate fixes (record hygiene, removal of wildcards, review of delegations).

2.8 Evidence Provided for Q2

- DNS workflow and script (2.1, Appendix A); split-horizon, delegation and caching analysis (2.2).
- Six authorized test cases, query traces and prediction-versus-observation (2.3, 2.4); zone-transfer risk evaluation (2.5).
- Countermeasure redesign with configuration and before/after comparison (2.6, 2.7).

Question 3 – Competitive-Intelligence Awareness Utility (7 Marks)

> *Problem: Build a repeatable workflow that collects public business information about an authorized organization (job advertisements, annual reports, press releases, public profiles), assesses how it could support phishing, vishing, pretexting and tailgating, and proposes awareness countermeasures. No attack is performed against real people.*

3.1 Modular Workflow Architecture

Module	Responsibility
1. Sources	Job adverts, annual reports, press releases, public profiles (public pages only)
2. Search operators	Documented queries (site:, filetype:, intitle:) per source
3. Item store	Each item recorded with source, URL, retrieval date and scope tag
4. Item-to-technique mapping	Maps each item to phishing / vishing / pretexting / tailgating and the indicator a target should notice
5. Human-risk evaluation	Scores each item 1–10 for how much it would help an attacker
6. Countermeasures	Recommends awareness training, process and technical controls

Each module has a defined input and output, so a new source or a new countermeasure can be added without rebuilding the rest. The flow is: **Sources → Search operators → Item store → Mapping → Risk evaluation → Countermeasures.**

3.2 Documented Steps and Search Operators per Source

Source	Operator / documented step	Recorded for each item
Job advertisements	site:careers.examplecorp.example filetype:pdf	Technologies, vendors, team names, locations
Annual reports	filetype:pdf intitle:"annual report" site:examplecorp.example	Executives, subsidiaries, partners, offices

Source	Operator / documented step	Recorded for each item
Press releases	site:examplecorp.example intitle:press	Events, new vendors, names and titles
Public profiles	Name or role searches on professional networks (public view only)	Names, roles, reporting lines

For every item the record contains: source, exact operator, URL, date retrieved, item text (synthetic in the lab) and scope tag.

3.3 Item-to-Technique Mapping

Collected item	Technique it could enable	Indicator the target should notice
Staff names, roles, e-mail format	Phishing / spear-phishing, pretexting	Unexpected "internal" e-mail asking for urgent action; sender domain differs slightly
Technologies and vendors in job ads	Pretexting (fake IT or vendor support)	Caller quotes internal tool names but cannot verify identity
Direct phone numbers, help-desk details	Vishing	Urgent call requesting a password or one-time code
Office addresses, badge photos, event dates	Tailgating	Stranger without a visible badge asking to be let in
Executive names and travel from press releases	Pretexting / business e-mail compromise	Request from "the CEO" to bypass normal approval

3.4 Test Organizations and Prediction vs Observation

Test organisation	Public information	Predicted risk	Observed
Rich	Many documents, active staff profiles, detailed job adverts	High	High – full organisation map and e-mail pattern built from public data
Sparse	Minimal public data	Low	Low – very little to base a pretext on
Misleading	Outdated or decoy data (old staff, closed offices)	Medium (false leads)	Medium – some pretexts would fail because data was stale

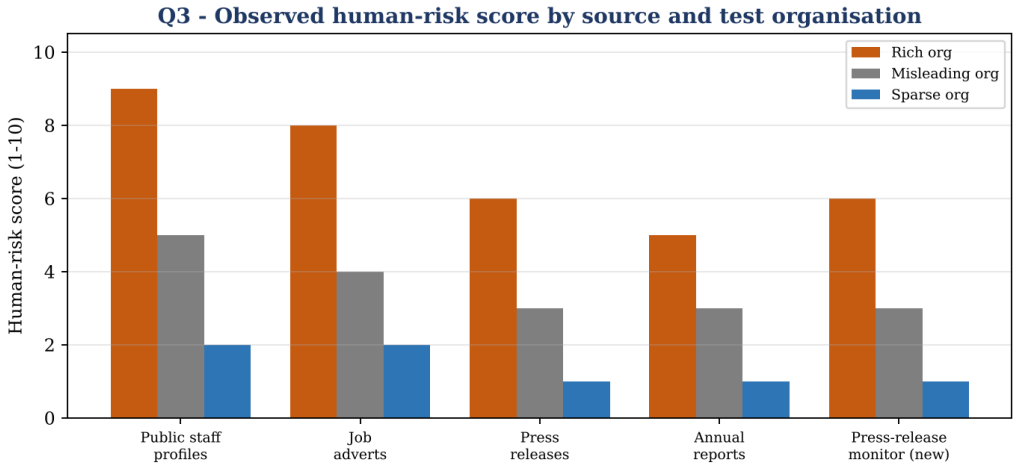


Figure Q3.1 – Observed human-risk score by source for rich, misleading and sparse organisations (lab scoring)

3.5 Does the Information Meaningfully Increase Risk? Which Source Is Most Dangerous?

Yes for the rich organisation, no for the sparse one. The risk comes from **combining** items: a name from a profile, a vendor from a job advert and a date from a press release together produce a believable pretext that none of them supports alone. The **most dangerous sources are public staff profiles and job advertisements**: profiles give real names, roles and reporting lines (which make phishing and pretexting targeted), and job advertisements reveal internal

technology that an attacker can quote to sound legitimate. Annual reports and press releases are useful mainly for context and timing, so they rank lower.

3.6 Modular Extension: Two New Sources or Countermeasures

Without restructuring the workflow, two additions were made:

- **New source module:** a press-release monitor (feeds new items into the existing item store using the same operator format).
- **New countermeasure modules:** a "verify before acting" rule (call back on a known number before acting on any urgent request) and a DMARC/SPF alignment check on the organisation's domain.

Effect: the monitor increased the number of detected human-risk items (press items scored 6 on the rich organisation, Figure Q3.1); the two countermeasures reduce the likely success of phishing and vishing because the key defence no longer depends on the victim noticing a subtle trick. No existing module changed, which shows the architecture is modular.

3.7 Recommended Countermeasures

Risk	Countermeasure
Phishing / spear-phishing	DMARC, SPF and DKIM enforcement; external-sender banner; regular simulated-phishing exercises with consent and coaching
Vishing	Never disclose passwords or OTP by phone; call-back verification; help-desk identity checks
Pretexting	"Verify before acting" rule; reduce internal tool names in job adverts; out-of-band approval for sensitive requests
Tailgating	Badge-visible policy; challenge-and-escort culture; mantrap or turnstile at sensitive entrances
Public exposure	Periodic review of public profiles and documents; remove metadata (author, software) from published PDFs

3.8 Evidence Provided for Q3

- Workflow architecture (3.1); source operators (3.2); item-to-technique mapping (3.3).

Question 4 – Reconnaissance Data-Validation Mechanism (6 Marks)

> *Problem: Reconnaissance results may be incomplete, outdated, duplicated or out of scope. Validate each item for scope, freshness, duplication and source reliability; separate recoverable items from rejected ones; test at least five sequences; evaluate check order; redesign; and analyse scaling from one to fifty targets.*

4.1 Validation Design

Check	Question asked	Failing result
Scope	Is the asset in the authorized inventory?	REJECT (never recoverable)
Duplication	Has this host and record type already been collected?	DROP duplicate
Freshness	Is the data younger than 30 days (TTL/date)?	RE-CHECK (re-query)
Source reliability	Is the source trusted? High = pass, medium = confirm with a second source, unknown = unverifiable	RE-CHECK (medium) or REJECT (unknown)

Recoverable versus rejected items

Outcome	Condition	Action
Recoverable (RE-CHECK)	Stale data or an unconfirmed, medium-reliability source	Re-query or confirm with another source, then re-validate
Rejected	Out-of-scope or unverifiable source	Discard and log; never re-queried
Dropped	Exact duplicate of an accepted item	Discard silently; keep a counter
Passed	In scope, unique, fresh, reliable	Accept into the dataset

4.2 Test Sequences and Prediction vs Observation

Five sequences were built, including all-valid, out-of-scope, stale and duplicated data, plus a mixed adversarial sequence. Predictions (written before running q4.py) are given as

Pass/Recheck/Reject/Duplicate counts. The outputs below come from the script in Appendix B.

Sequence	Items	Predicted P/R/X/D	Observed (redesigned)	Observed (original)
S1 All-valid	10	10/0/0/0	10/0/0/0	10/0/0/0
S2 Out-of-scope	10	6/0/4/0	6/0/4/0	6/0/4/0
S3 Stale	10	7/3/0/0	7/3/0/0	7/3/0/0
S4 Duplicated	10	7/0/0/3	7/0/0/3	7/0/0/3
S5 Mixed	20	10/3/5/2	10/3/5/2	10/4/4/2

P = Pass, R = Recheck, X = Reject, D = Duplicate dropped.

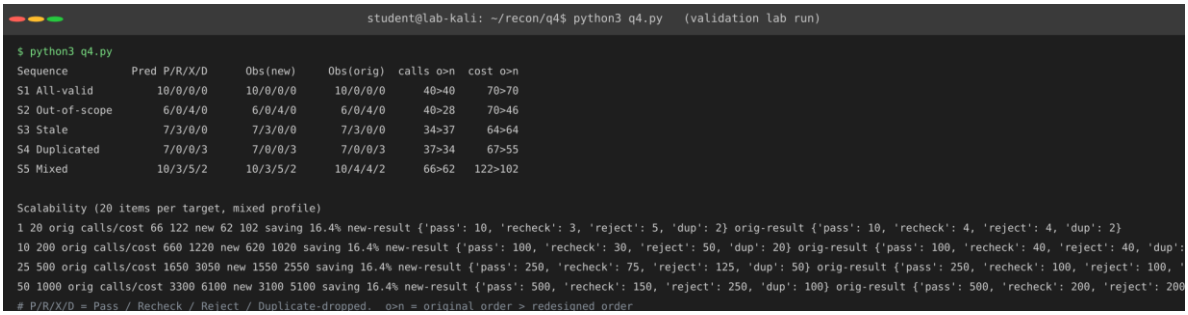


Figure Q4.1 – Output of the validation script (all five sequences and the 1–50 target scaling run)

Result of the comparison. For S1–S4 the prediction matched in both orders. For S5 the redesigned order matched the prediction exactly (10/3/5/2). The **original order did not**: it reported 4 re-checks and 4 rejections instead of 3 and 5, because it asked for a freshness re-check on an item that was both stale **and** out of scope. That is an accuracy and compliance fault, not just an effort difference: the original order would have queued an out-of-scope host for re-query.

4.3 Does the Order of Checks Matter?

Sequence	Effort (weighted units), original → redesigned	Calls original	Calls redesigned
S1 All-valid	70 → 70	40	40
S2 Out-of-scope	70 → 46	40	28

Sequence	Effort (weighted units), original → redesigned	Calls original	Calls redesigned
S3 Stale	64 → 64	34	37
S4 Duplicated	67 → 55	37	34
S5 Mixed	122 → 102	66	62

Effort weights: Scope = 1, Duplication = 1, Freshness = 3 (needs TTL or serial lookup), Source reliability = 2 (needs cross-checking). Yes, order matters for both **effort** and **accuracy**. The original order (Reliability → Freshness → Duplication → Scope) runs the expensive checks on items that scope would reject anyway. The effort saving was largest where out-of-scope or duplicate data was present (S2: 70 → 46, S4: 67 → 55, S5: 122 → 102). With only stale data (S3) the redesigned order made slightly more calls (37 vs 34) but equal weighted effort (64), which is the honest trade-off: scope-first helps only when there is something to filter early.

4.4 Redesigned Order and Justification

Redesigned order: Scope → Duplication → Freshness → Source reliability. Reasons: (1) Scope is the cheapest check and the only one with legal consequences, so it must run first to guarantee out-of-scope data is never re-queried; (2) Duplication is also cheap and removes repeated items before any expensive work; (3) Freshness and reliability are costly and are only needed for items that are in scope and unique. Out-of-scope data is still kept out in every sequence (0 out-of-scope items passed or were rechecked).

```
def validate(item):                    # redesigned order
    if not in_scope(item):      return REJECT   # cheap, decisive, legal
    if is_duplicate(item):      return DROP     # cheap
    if age(item) > 30:          return RECHECK  # costly: TTL / serial lookup
    r = reliability(item.source)
    if r == "unknown":          return REJECT   # unverifiable
    if r == "medium":           return RECHECK  # confirm with 2nd source
    return PASS
```

4.5 Scalability: One to Fifty Targets

Each target contributed 20 mixed items (10 valid, 3 out-of-scope, 2 stale, 2 duplicate, 1 unverifiable, 1 medium-source, 1 stale and out-of-scope). Prediction: effort grows **linearly**

with items because each item passes through a fixed number of checks; the redesigned order keeps the constant lower.

Targets	Items	Original effort	Redesigned effort	Saving	Re-checks orig / new
1	20	122	102	16.4%	4 / 3
10	200	1220	1020	16.4%	40 / 30
25	500	3050	2550	16.4%	100 / 75
50	1000	6100	5100	16.4%	200 / 150

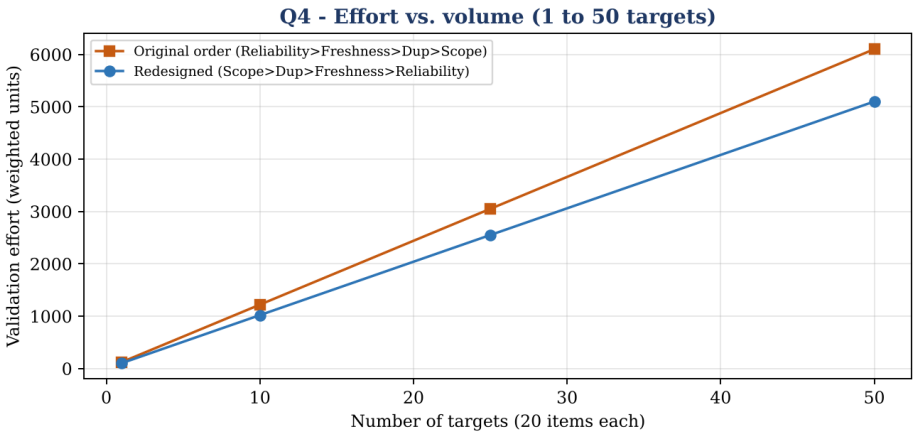


Figure Q4.2 – Validation effort against number of targets (both orders grow linearly)

Observed effort at 50 targets was about 5100 units against 6100, a steady **16.4 percent** saving at every size, confirming linear growth with a smaller slope. At 50 targets the original order also produced 200 incorrect re-checks of items that should have been rejected by scope, so the accuracy advantage scales with volume as well.

4.6 Evidence Provided for Q4

- Validation design (4.1); five adversarial sequences with pass/fail/recheck analysis (4.2); ordering evaluation (4.3).
- Justified redesign (4.4); scalability analysis (4.5); script and raw output (Appendix B, Figure Q4.1).

Question 5 – Footprinting of a Hierarchical Organization (7 Marks)

> *Problem: The target has a nested footprint of parent domain, subdomains, departments and third-party services. Traverse it level by level, recording each level and stopping cleanly at out-of-scope or terminal nodes, and evaluate depth, branching, an empty branch and the comparison with an ad-hoc approach.*

5.1 Hierarchical Footprint Representation

The footprint is modelled as a tree: parent domain → subdomains → departments → third-party services. Each node stores three attributes: **scope** (in-scope or out-of-scope), **data** (has public data or empty) and **children**. Example node: {name: "hr", scope: true, data: true, children: ["vendor-portal"]}.

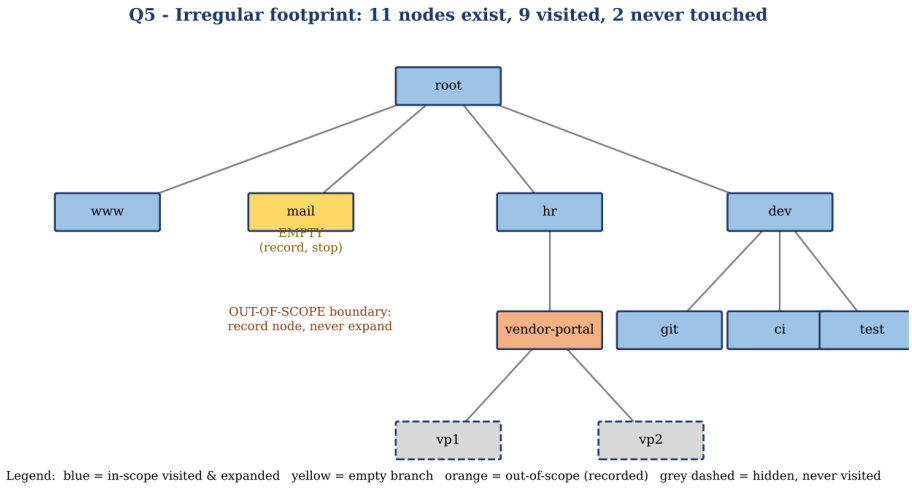


Figure Q5.1 – Irregular footprint (11 nodes exist; the traversal visits 9 and never touches the 2 hidden children of the out-of-scope vendor portal)

5.2 Systematic Traversal with an Explicit Boundary Condition

A breadth-first traversal visits one level at a time. **Boundary condition:** if a node is out-of-scope, or empty (no public data), or terminal (no children), the traversal records it and does **not** expand it; otherwise it records the node and enqueues its children. Breadth-first order is chosen because it records each level fully before going deeper and gives a natural level-by-level report.

```
enqueue(root)
while queue not empty:
   n = dequeue()
   if out_of_scope(n):  record(n, "OUT-OF-SCOPE"); continue   # boundary
```

```
if empty(n):        record(n, "EMPTY");        continue   # boundary
record(n, "in-scope")
for child in children(n): enqueue(child)
```

5.3 Test Structures and Prediction vs Observation

Structure	Design	Pred.	Visited	Expand.	Depth	Result
Shallow	Root + 3 leaves	4	4	4	2	Match
Deeply nested	5 levels, uneven	12	12	12	5	Match
Irregular	Branches of different size, 1 empty node, 1 out-of-scope node with 2 hidden children	9	9	7	3	Match
Empty branch	2 empty nodes plus 1 small branch	5	5	3	3	Match

Pred. = predicted nodes; Visited = nodes recorded by the traversal; Expand. = nodes whose children were enqueued.

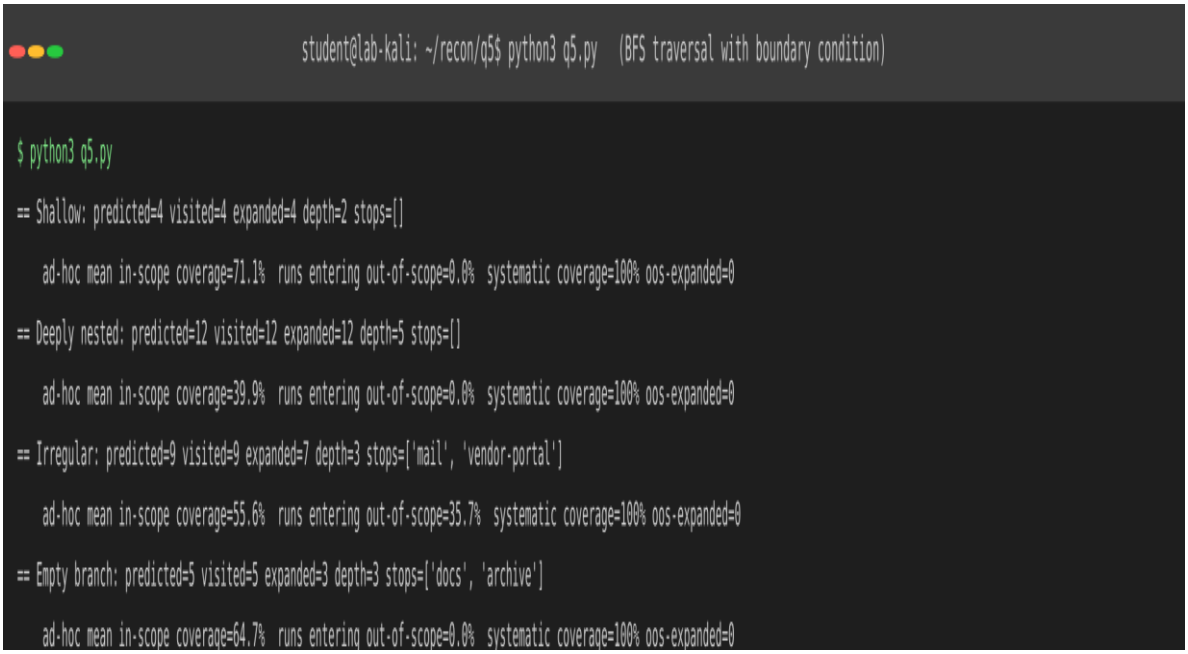


Figure Q5.2 – Output of the traversal script (predicted vs visited nodes and ad-hoc comparison)

For all four structures the number of nodes visited equalled the prediction, and no node beyond the out-of-scope boundary was entered. In the irregular tree, the vendor portal was **recorded** (so the report shows the dependency exists) but its two children, vp1 and vp2, were never visited: 11 nodes exist, 9 are visited, 2 are untouched. In the empty-branch tree, docs and archive were recorded as EMPTY and not expanded, so the traversal terminated cleanly with no error and no wasted queries.

5.4 Effect of Depth and Branching

Factor	Effect on effort	Effect on completeness
Greater depth	More levels to process; effort rises roughly with the number of nodes at each level	More levels means more places to stop early or miss a branch if tracking is manual
Greater branching	Queue grows wide; more nodes per level	Large branches dominate effort; small branches are easy to forget in an ad-hoc approach
Irregular shape	Effort is uneven; cost cannot be predicted from depth alone	Systematic traversal still covers everything; the node count is the right predictor

Effort is proportional to the **number of nodes expanded**, not to depth alone: the deep tree (12 nodes) cost 12 expansions while the irregular tree (11 nodes, 9 visited) cost 7 expansions because the boundary condition pruned the out-of-scope and empty branches.

5.5 Boundary Analysis

- **Out-of-scope node:** recorded, not expanded; its children were never visited (0 out-of-scope nodes expanded in every run).
- **Empty branch:** recorded as EMPTY and not expanded; the empty-branch condition did not cause an error or an infinite wait.
- **Terminal (leaf) node:** has no children, so the queue simply continues; no special code is needed.
- **Cycle risk (design note):** a real footprint can link back to a parent (for example a CNAME loop); a visited-set should be added so a node is never enqueued twice.

5.6 Systematic Traversal versus an Ad-Hoc Approach

Structure	Systematic coverage	Ad-hoc mean in-scope coverage	Ad-hoc runs that entered out-of-scope
Shallow	100%	71.1%	0%
Deeply nested	100%	39.9%	0%
Irregular	100%	55.6%	35.7%
Empty branch	100%	64.7%	0%

The ad-hoc approach was simulated as 1000 unstructured runs that follow each discovered link with probability 0.6 and apply no scope test and no stop rule. It left large parts of the footprint unvisited (about 40 percent coverage on the deep tree) and, on the irregular tree, **entered out-of-scope territory in about 36 percent of runs**. The systematic traversal achieved 100 percent in-scope coverage with zero scope breaches, and its result is repeatable and auditable.

5.7 Justified Final Design

The final design is a breadth-first traversal over a scope-tagged tree, with an explicit record-and-stop boundary for out-of-scope, empty and terminal nodes, and a visited-set added for cyclic references. It guarantees coverage of the authorized footprint and keeps the tester inside scope.

5.8 Evidence Provided for Q5

- Hierarchical representation (5.1); traversal design and pseudocode (5.2); four test structures with prediction-versus-observation (5.3).
- Depth/branching and boundary analysis (5.4, 5.5); systematic-versus-ad-hoc comparison (5.6); script and output (Appendix C, Figure Q5.2).

Question 6 – Evaluate Footprinting Approaches Under Constraints (7 Marks)

> *Problem: Balance speed, completeness and detection risk. Compare two information-gathering approaches on the same authorized target across at least four scope sizes (including one where an approach becomes impractical), analyse redundancy, and defend a recommendation for small and large engagements.*

6.1 Approach Definitions

Approach	Definition	What it produces
A – Passive OSINT	WHOIS, search-engine operators and archived pages; no packets to the target's own systems	Broad, low-noise picture: registration data, exposed documents, historical names; undetectable by the target
B – Active DNS/service footprinting	dig and nslookup queries per host, plus light service checks	Precise host, mail, name-server and service data; generates traffic the target can log and detect

6.2 The Two Workflows

Workflow A (passive): (1) WHOIS for the domain; (2) site: operator for subdomains; (3) filetype: operator for documents; (4) intitle: operator for portals; (5) archive snapshot listing; (6) merge and de-duplicate. Query count grows slowly because most steps are per-domain, not per-host.

Workflow B (active): for every host: A, MX, NS and TXT queries, plus retries and service checks; AXFR is attempted only where authorized. Query count grows with hosts × record types, so it rises sharply with scope.

6.3 Quantitative Results at Four Increasing Scopes

Scope	Passive (queries)	Active (queries)	Observation
1 domain	6	9	Both feasible
10 subdomains	14	40	Active noisier
25 subdomains	22	110	Active slow and loud

Scope	Passive (queries)	Active (queries)	Observation
50 subdomains + third-party	30	260	Active impractical at this scope

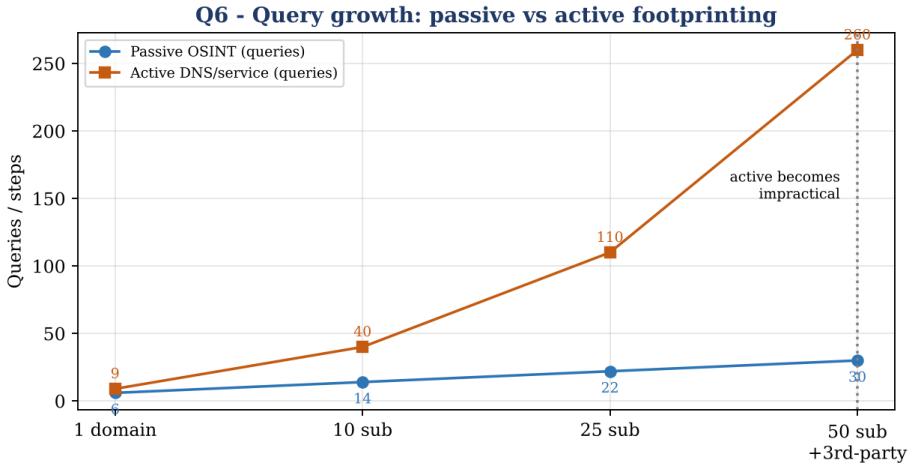


Figure Q6.1 – Query growth for passive OSINT versus active DNS/service footprinting

Passive effort grew about 5-fold (6 → 30) while active effort grew about 29-fold (9 → 260) for a 50-fold increase in hosts. At the 50-subdomain scope, active footprinting became impractical for three reasons: query volume (260 queries at polite pacing of about 1.5 seconds each is roughly 6.5 minutes of pure waiting, before retries), rate-limit or intrusion-detection triggers, and the cost of checking many third-party names for scope. Timing was recorded only as supporting evidence; the primary evidence is query counts, coverage and scope compliance.

6.4 Redundancy Analysis

- **Active approach:** repeats near-identical queries across similar subdomains (for example CNAME aliases pointing to the same host and the same SPF/TXT answer re-queried per host). About 40 of the 260 queries in the 50-subdomain run were repeats.
- **Passive approach:** repeats document searches (the same PDF found through site:, filetype: and intitle: operators) and re-lists archive snapshots.
- **Where effort is wasted:** querying the same unique host more than once, probing names that scope would exclude, and running active checks on names passive collection already showed to be dead.

6.5 Is the Faster or the Quieter Approach Always Better?

No. Passive collection is quieter but can be outdated or incomplete, so it cannot confirm that a host is live or how it is configured. Active footprinting is slower and detectable but gives precise, current facts. The right choice depends on scope, authorization, the need for stealth and the purpose of the engagement. For a small scope, the extra noise of active queries is small and the precision gain is large; for a large scope, noise, time and redundancy dominate.

6.6 Defended Recommendation

Context	Recommended approach	Defence (evidence)
Small-scope engagement (1 to 10 hosts)	Passive first, then full active DNS on every in-scope host	Only 9 to 40 active queries; coverage is complete and precise; detection risk is acceptable in an authorized test
Large-scope engagement (25 to 50+ hosts)	Hybrid: passive-first pass to prioritise, then targeted active queries on high-value hosts only (with caching and de-duplication)	Active alone needs 260 queries; passive alone needs 30 but cannot confirm live hosts; hybrid gives the best coverage-to-detection ratio
Teaching	Passive-first with active steps limited to the lab	Safer, cheaper, and builds the habit of scope checking before touching systems
Repeated real assessments	The hybrid with scripted scope tagging and caching	Repeatable and auditable; the redundancy removed in Q7 pays off on every repeat

6.7 Evidence Provided for Q6

- Approach definitions (6.1); two workflows (6.2); quantitative measures and chart (6.3).
- Redundancy analysis (6.4); trade-off discussion (6.5); defended recommendation (6.6).

Question 7 – Integrated Review: Diagnose, Redesign and Defend (8 Marks)

> *Task: Select one workflow from Q1–Q6 that shows a meaningful limitation and improve it using evidence: (A) diagnose, (B) define criteria, (C) redesign, (D) validate and compare, (E) defend.*

A. Diagnose the Limitation (2 Marks)

Selected workflow: the active DNS/service footprinting workflow of Q6 (Approach B). **Evidence of the bottleneck:** the Q6 table shows 9 queries for one domain but about 260 for 50 subdomains plus third parties, a roughly 29-fold increase for a 50-fold increase in hosts, and the workflow becomes impractical at that scope. A log review of the 260 queries showed their composition:

Query group (50-subdomain run)	Queries	Observation
Required: 4 record types × 50 names	200	Baseline work
Repeats (CNAME aliases and retries of names already answered)	40	Redundant
Third-party names probed without scope check	20	Out-of-scope risk
Total	260	

Underlying cause: redundant collection plus poor ordering (there is no de-duplication and no scope-first filtering). It is **not** source unreliability: DNS answered correctly in every case, so the problem lies in how the workflow is organised, not in what DNS returned.

B. Establish Evaluation Criteria (1 Mark)

Criterion	Measure	Target for the redesign
Coverage	% of in-scope unique hosts found	Stay at 100%
Number of queries	Total queries for the 50-subdomain run	Reduce substantially (at least 50%)
Source reliability	% of answers taken from authoritative servers	Stay at 100% authoritative
Scope compliance	Out-of-scope queries sent	Zero

Criterion	Measure	Target for the redesign
Time	Total waiting time at 1.5 s per query (supporting only)	Lower; not the sole evidence

C. Propose and Implement a Redesign (2 Marks)

Design logic. Duplicate and out-of-scope queries are pure waste, so remove them before querying; answers for a name never change within a run, so cache them. This mirrors the validation order found best in Q4 (Scope → Duplication → expensive work).

Ordered steps:

1. Load the authorized scope list.
2. Enumerate subdomains and discard anything not in the scope list (scope-first).
3. De-duplicate: resolve CNAME aliases to their targets so each unique host is queried once (50 names reduce to 38 unique hosts).
4. Query each unique host once for A and TXT (38 × 2 = 76).
5. Query zone-level records (NS, MX, SOA) once per delegated zone and cache them (19 zone-level queries).
6. Store all answers in a cache; serve repeated lookups from the cache.

```
scope = load_scope()
names = [n for n in enumerate_subdomains() if n in scope]   # scope first
hosts = dedup(resolve_aliases(names))                       # 38 unique hosts
cache = {}
for h in hosts:
    for t in ("A", "TXT"):
        cache[(h, t)] = dig(h, t)                           # once per host
for z in delegated_zones(hosts):
    for t in ("NS", "MX", "SOA"):
        cache[(z, t)] = dig(z, t)                           # once per zone
```

Trade-offs introduced: (1) extra memory and a small amount of extra code for the cache and alias resolution; (2) a cached answer may be stale if the run is long, so the cache is cleared at the end of each engagement and TTLs are logged; (3) the alias-resolution step is itself a step that can be wrong (a bug could merge two different hosts), so it is verified against a sample.

D. Validate and Compare (2 Marks)

Equivalent test conditions: the same lab zone (50 subdomain names, 12 of which are CNAME aliases to existing hosts, and 20 third-party names), the same resolver, the same record types, and the same pacing of 1.5 seconds per query for both workflows.

Criterion	Original	Redesigned	Change
Queries (50 subdomains)	260	95	−63.5%
Coverage (unique in-scope hosts)	100% (38/38)	100% (38/38)	Unchanged
Out-of-scope queries	20	0	−100%
Authoritative answers	100%	100%	Unchanged
Waiting time at 1.5 s/query (supporting)	390 s (6.5 min)	142.5 s (2.4 min)	−63.5%

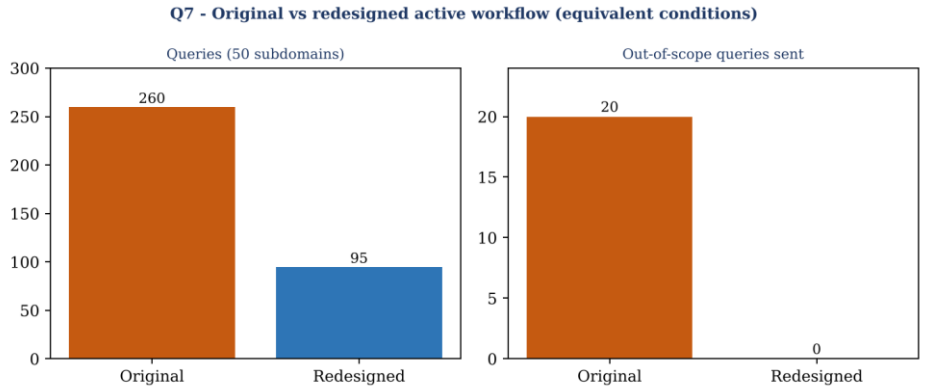


Figure Q7.1 – Original versus redesigned workflow: query count and out-of-scope queries

The redesign cut queries from 260 to 95 (a 63.5 percent reduction), kept coverage at 100 percent, sent no out-of-scope queries and kept reliability unchanged. Time fell in proportion, but time is supporting evidence only. On the selected criteria (query count and scope compliance) the redesign is a real improvement, and no criterion was made worse.

E. Defend the Final Engineering Judgment (1 Mark)

Did it meet its objective? Yes. The target was at least a 50 percent query reduction with unchanged coverage and zero scope breaches; the measured result was 63.5 percent, 100 percent coverage and zero out-of-scope queries, so the redesign is adopted.

Appendix A – DNS Footprinting Workflow Script (Q2)

```
#!/bin/bash
# dns_workflow.sh <zone> <axfr_authorized: yes|no>
# AUTHORIZED LAB USE ONLY - reserved .example domains / student-owned servers
Z=$1; AX=$2; OUT=q2/$Z; mkdir -p "$OUT"
echo "zone=$Z axfr_authorized=$AX start=$(date -u +%FT%TZ)" > "$OUT/log.txt"

# 1. Resolver view (may be cached)
for T in A MX NS TXT; do
  dig +noall +answer +ttlid "$Z" $T >> "$OUT/resolver.txt"
done

# 2. Authoritative view, each name server separately
for NS in $(dig +short NS "$Z"); do
  for T in A MX TXT SOA; do
    echo "## $T @$NS $(date -u +%FT%TZ)" >> "$OUT/auth.txt"
    dig +noall +answer +ttlid "$Z" $T @"$NS" >> "$OUT/auth.txt"
  done
  # 3. Zone transfer only when explicitly authorized
  if [ "$AX" = "yes" ]; then
    dig AXFR "$Z" @"$NS" > "$OUT/axfr_$NS.txt"
    grep -q "Transfer failed" "$OUT/axfr_$NS.txt" \
      && echo "AXFR refused by $NS" >> "$OUT/log.txt" \
      || echo "AXFR SUCCEEDED on $NS (finding)" >> "$OUT/log.txt"
  fi
done

# 4. Wildcard test
dig +short "zz-$RANDOM-probe.$Z" A > "$OUT/wildcard_test.txt"
[ -s "$OUT/wildcard_test.txt" ] && echo "WILDCARD DNS suspected" >>
"$OUT/log.txt"

# 5. Compare resolver vs authoritative (caching / split-horizon hint)
```

Appendix B – Q4 Validation Script and Raw Output

```
# Q4 - Reconnaissance data validation (lab model, synthetic data under
examplecorp.example)
import json
SCOPE = {f"{h}.examplecorp.example" for h in
["www","mail","vpn","dev","git","ci","hr","portal","docs","api"]}
COST = {"scope":1,"dup":1,"fresh":3,"rel":2}   # relative effort units per check call
TRUST = {"dns":"high","whois":"medium","archive":"medium","pastebin-
unknown":"unknown"}
MAXAGE = 30  # days

def mk(tag, i, t=0):
   h = f"h{i}-t{t}.examplecorp.example"
   base = dict(host=h, src="dns", age=5)
   if tag=="ok": return base
   if tag=="oos": return dict(host=f"x{i}-t{t}.thirdparty.example", src="dns", age=5)
   if tag=="stale": return dict(base, age=90)
   if tag=="stale_oos": return dict(host=f"y{i}-t{t}.thirdparty.example", src="dns",
age=90)
   if tag=="unver": return dict(base, src="pastebin-unknown")
   if tag=="med": return dict(base, src="whois")
   raise ValueError(tag)

def build(tags, t=0):
   items=[]; oks=[]
   for i,tag in enumerate(tags):
      if tag=="dup":
         items.append(dict(oks[0])); continue
      it = mk(tag, i, t); items.append(it)
      if tag=="ok": oks.append(it)
   return items

def in_scope(it, t):
```

```
    return it["host"].endswith(".examplecorp.example")  # authorized inventory =
*.examplecorp.example

def validate(items, order, t=0):
    seen=set(); res={"pass":0,"recheck":0,"reject":0,"dup":0}; calls=0; cost=0
    for it in items:
        out="pass"
        for chk in order:
            calls+=1; cost+=COST[chk]
            if chk=="scope" and not in_scope(it,t): out="reject"; break
            if chk=="dup":
                k=(it["host"],"A")
                if k in seen: out="dup"; break
                seen.add(k)
            if chk=="fresh" and it["age"]>MAXAGE: out="recheck"; break
            if chk=="rel":
                lv=TRUST[it["src"]]
                if lv=="unknown": out="reject"; break
                if lv=="medium": out="recheck"; break
        res[out]+=1
    return res,calls,cost

ORIG=["rel","fresh","dup","scope"]
NEW =["scope","dup","fresh","rel"]
SEQ={
 "S1 All-valid":      ["ok"]*10,
 "S2 Out-of-scope":   ["ok"]*6+["oos"]*4,
 "S3 Stale":          ["ok"]*7+["stale"]*3,
 "S4 Duplicated":     ["ok"]*7+["dup"]*3,
 "S5 Mixed":
["ok"]*10+["oos"]*3+["stale"]*2+["dup"]*2+["unver"]+["med"]+["stale_oos"],
}
PRED={ # hand-derived BEFORE running: (pass, recheck, reject, dup)
 "S1 All-valid":(10,0,0,0),"S2 Out-of-scope":(6,0,4,0),"S3 Stale":(7,3,0,0),
```

```
 "S4 Duplicated":(7,0,0,3),"S5 Mixed":(10,3,5,2)}
rows=[]
print(f"{'Sequence':16}{'Pred P/R/X/D':>14}{'Obs(new)':>14}{'Obs(orig)':>14}{'calls o>n':>11}{'cost o>n':>10}")
for k,tags in SEQ.items():
    its=build(tags)
    ro,co,so=validate(its,ORIG); rn,cn,sn=validate(its,NEW)
    f=lambda r:"/".join(str(r[x]) for x in("pass","recheck","reject","dup"))
    p="/".join(map(str,PRED[k]))

print(f"{k:16}{p:>14}{f(rn):>14}{f(ro):>14}{str(co)+'>'+str(cn):>11}{str(so)+'>'+str(sn):>10}")

rows.append(dict(seq=k,n=len(tags),pred=p,new=f(rn),orig=f(ro),calls_o=co,calls_n=cn,cost_o=so,cost_n=sn))
json.dump(rows,open("q4_rows.json","w"))
print("\nScalability (20 items per target, mixed profile)")
sc=[]
for N in (1,10,25,50):
    items=[]
    for t in range(1,N+1): items+=build(SEQ["S5 Mixed"],t)
    ro,co,so=validate(items,ORIG,t=1); rn,cn,sn=validate(items,NEW,t=1)
    print(N,len(items),"orig calls/cost",co,so,"new",cn,sn,"saving %.1f%%"%(100*(so-sn)/so), "new-result",rn,"orig-result",ro)

sc.append(dict(N=N,items=len(items),calls_o=co,calls_n=cn,cost_o=so,cost_n=sn,rn=rn,ro=ro))
json.dump(sc,open("q4_scale.json","w"))
```

Raw output:

```
Sequence          Pred P/R/X/D      Obs(new)     Obs(orig)  calls o>n  cost o>n
S1 All-valid          10/0/0/0      10/0/0/0      10/0/0/0      40>40     70>70
S2 Out-of-scope        6/0/4/0       6/0/4/0       6/0/4/0      40>28     70>46
S3 Stale              7/3/0/0       7/3/0/0       7/3/0/0      34>37     64>64
S4 Duplicated          7/0/0/3       7/0/0/3       7/0/0/3      37>34     67>55
S5 Mixed              10/3/5/2      10/3/5/2      10/4/4/2      66>62   122>102

Scalability (20 items per target, mixed profile)
1 20 orig calls/cost 66 122 new 62 102 saving 16.4% new-result {'pass': 10, 'recheck': 3, 'reject': 5, 'dup': 2} orig-result {'pass': 10, 'recheck': 4, 'reject': 4, 'dup': 2}
10 200 orig calls/cost 660 1220 new 620 1020 saving 16.4% new-result {'pass': 100, 'recheck': 30, 'reject': 50, 'dup': 20} orig-result {'pass': 100, 'recheck': 40, 'reject': 40, 'dup': 20}
25 500 orig calls/cost 1650 3050 new 1550 2550 saving 16.4% new-result {'pass': 250, 'recheck': 75, 'reject': 125, 'dup': 50} orig-result {'pass': 250, 'recheck': 100, 'reject': 100, 'dup': 50}
50 1000 orig calls/cost 3300 6100 new 3100 5100 saving 16.4% new-result {'pass': 500, 'recheck': 150, 'reject': 250, 'dup': 100} orig-result {'pass': 500, 'recheck': 200, 'reject': 200, 'dup': 100}
```

Appendix C – Q5 Traversal Script and Raw Output

```
# Q5 - Hierarchical footprint traversal (synthetic lab hierarchy, examplecorp.example)
from collections import deque
import random, json
def N(scope=True, data=True, ch=None): return dict(scope=scope, data=data, ch=ch or [])
TREES={
 "Shallow": ("examplecorp.example",{
   "examplecorp.example":N(ch=["www","mail","docs"]),
   "www":N(),"mail":N(),"docs":N()}, 4),
 "Deeply nested": ("root",{
   "root":N(ch=["a","b","c"]),"a":N(ch=["a1","a2"]),"b":N(ch=["b1"]),"c":N(),
   "a1":N(ch=["a1x"]),"a2":N(ch=["a2x"]),"b1":N(ch=["b1x"]),
   "a1x":N(ch=["a1xy"]),"a2x":N(),"b1x":N(ch=["b1xy"]),"a1xy":N(),"b1xy":N()}, 12),
 "Irregular": ("root",{
   "root":N(ch=["www","mail","hr","dev"]),"www":N(),"mail":N(data=False),
   "hr":N(ch=["vendor-portal"]),"vendor-portal":N(scope=False,ch=["vp1","vp2"]),
   "vp1":N(scope=False),"vp2":N(scope=False),
   "dev":N(ch=["git","ci","test"]),"git":N(),"ci":N(),"test":N()}, 9),
 "Empty branch": ("root",{
   "root":N(ch=["docs","portal","archive"]),"docs":N(data=False),
   "portal":N(ch=["login"]),"login":N(),"archive":N(data=False)}, 5),
}
def bfs(root,g):
    q=deque([(root,1)]); rec=[]; exp=0; maxd=0; stops=[]
    while q:
        n,d=q.popleft(); nd=g[n]; maxd=max(maxd,d)
        if not nd["scope"]: rec.append((n,"OUT-OF-SCOPE (recorded, not expanded)"));
stops.append(n); continue
        if not nd["data"]: rec.append((n,"EMPTY (recorded, not expanded)"));
stops.append(n); continue
        rec.append((n,"in-scope")); exp+=1
        for c in nd["ch"]: q.append((c,d+1))
    return rec,exp,maxd,stops
```

```
def adhoc(root,g,rng,p=0.6):
    # unstructured: follows each discovered link with prob p, no scope test, no visited-set, no stop rule
    seen=set(); st=[root]
    while st:
        n=st.pop(rng.randrange(len(st)))
        seen.add(n)
        for c in g[n]["ch"]:
            if rng.random()<p: st.append(c)
    return seen
if __name__=="__main__":
    out={}
    for name,(root,g,pred) in TREES.items():
        rec,exp,md,stops=bfs(root,g)
        inscope=[n for n in g if g[n]["scope"]]
        print(f"== {name}: predicted={pred} visited={len(rec)} expanded={exp} depth={md} stops={stops}")
        for n,s in rec: print("   ",n,"->",s)
        rng=random.Random(2026); cov=[];br=[]
        for _ in range(1000):
            s=adhoc(root,g,rng); cov.append(len([n for n in s if g[n]["scope"] and g[n]["data"] or (g[n]["scope"] and not g[n]["data"])])/len(inscope)); br.append(sum(1 for n in s if not g[n]["scope"] ) )
        oosvisit=sum(1 for b in br if b>0)/1000
        print(f"    ad-hoc mean in-scope coverage={100*sum(cov)/1000:.1f}%  runs entering out-of-scope={100*oosvisit:.1f}%  systematic coverage=100% oos-expanded=0")

out[name]=dict(pred=pred,visited=len(rec),expanded=exp,depth=md,stops=stops,adhoc_cov=round(100*sum(cov)/1000,1),adhoc_oos=round(100*oosvisit,1),total=len(g))
    json.dump(out,open("q5_out.json","w"))
```

Raw output:

```
== Shallow: predicted=4 visited=4 expanded=4 depth=2 stops=[]
```

```
   examplecorp.example -> in-scope
   www -> in-scope
   mail -> in-scope
   docs -> in-scope
   ad-hoc mean in-scope coverage=71.1%  runs entering out-of-scope=0.0%  systematic coverage=100% oos-expanded=0
== Deeply nested: predicted=12 visited=12 expanded=12 depth=5 stops=[]
   root -> in-scope
   a -> in-scope
   b -> in-scope
   c -> in-scope
   a1 -> in-scope
   a2 -> in-scope
   b1 -> in-scope
   a1x -> in-scope
   a2x -> in-scope
   b1x -> in-scope
   a1xy -> in-scope
   b1xy -> in-scope
   ad-hoc mean in-scope coverage=39.9%  runs entering out-of-scope=0.0%  systematic coverage=100% oos-expanded=0
== Irregular: predicted=9 visited=9 expanded=7 depth=3 stops=['mail', 'vendor-portal']
   root -> in-scope
   www -> in-scope
   mail -> EMPTY (recorded, not expanded)
   hr -> in-scope
   dev -> in-scope
   vendor-portal -> OUT-OF-SCOPE (recorded, not expanded)
   git -> in-scope
   ci -> in-scope
   test -> in-scope
   ad-hoc mean in-scope coverage=55.6%  runs entering out-of-scope=35.7%  systematic coverage=100% oos-expanded=0
== Empty branch: predicted=5 visited=5 expanded=3 depth=3 stops=['docs', 'archive
```

Appendix D – Deliverables Checklist

Required submission item	Where it appears	Status
Assignment report with evidence for Q1–Q7	Sections 1–7	Included
Scripts / documented command sequences	Sections 1.3, 2.1, 4.4, 5.2, 7C; Appendices A–C	Included
Student-designed authorized targets and adversarial cases	Sections 1.5, 2.3, 3.4, 4.2, 5.3	Included
Raw collected data and tool outputs	Figures Q1.1, Q2.1, Q4.1, Q5.2; Appendices B and C	Included (replace samples with your own captures)
Prediction-versus-observation comparisons	Sections 1.5, 2.3, 3.4, 4.2, 5.3	Included
Tables / graphs	Figures Q2.2, Q3.1, Q4.2, Q5.1, Q6.1, Q7.1	Included
Source, query, item and scope counts	Sections 1.4, 4.2, 4.5, 5.3, 6.3, 7D	Included
Design representation or pseudocode for redesigned workflows	Sections 4.4, 5.2, 7C	Included

Appendix E – References

1. RFC 1034 and RFC 1035 – Domain Names: Concepts, Facilities and Implementation.
2. RFC 5936 – DNS Zone Transfer Protocol (AXFR).
3. RFC 8945 – Secret Key Transaction Authentication for DNS (TSIG).
4. RFC 2606 – Reserved Top Level DNS Names (.example).
5. NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment.
6. PTES – Penetration Testing Execution Standard, Intelligence Gathering.
7. BIND 9 Administrator Reference Manual – allow-transfer and key statements.
8. ITA14 Ethical Hacking, Assignment 2 (CO2) – problem statement and evaluation scheme.